

Mutation

- **Riverpod 3: The Emergence of Mutation**
- More than just a new API; an evolution of the framework
- From “**State Management**” to “**Lifecycle Management**”

Paradigm Shift

- Up to Riverpod 2
 - **state itself** was at the center of management
- Core question of Notifier / AsyncNotifier
 - **What is this state right now?**
- Riverpod3
 - Evolving into “**Lifecycle Management of State Changes**”

Manual Management in Riverpod 2

- Process when a main action (e.g., **Create Todo**) is triggered
 - On Action execution → Manually change state to **loading**
 - On Success → Manually change state to **data**
 - On Failure → Manually change state to **error**
- **Problem:** Developers had to control every stage of state change through code

Repetition of Non-Essential Tasks

- The burden of managing all lifecycles accompanying an action fell on the developer
- Increase in **state control code** rather than essential business logic
- Led to inefficiency in code maintenance and boilerplate production

The Riverpod 3 Solution

- Focus strictly on the **Essential Action**
- Let the Framework automatically manage the lifecycle of state changes
- Developers focus on **what to do** and delegate the rest to the framework

Redefining the Core Concept

- What we were actually managing until Riverpod 2 was the **Lifecycle of the Action**
- Example: Thought process for **Create New Todo**
 - I need to change it to loading now.
 - It succeeded, so I'll put the data in.
 - If failed, so I'll show the error.

Why Mutation was Born

- Developers focus solely on the essence of **What to do**
- Framework takes responsibility for
 - Pending
 - Success
 - Failure
- Mutation doesn't replace state
 - it **separates and manages the action's process**

What is Mutation?

- Not a tool for representing state itself
- A tool that wraps and manages an **Intended Action** that changes state
- Encapsulates the entire **execution process**, not just the result

Complexity inside Notifier (The Legacy Way)

- When Add, Edit, and Delete logic are all in one Notifier:
 - Need to design a loading state for each action
 - Manually manage error exposure for each failure
- A structure where the developer is responsible for the entire action lifecycle

Mutation: The Action Wrapper

- Developer's role:
 - Tell Mutation to **Execute the action** and **Here's the success result**
- Mutation's role:
 - Automatically manage status, success results, and errors
- **Result:**
 - Freedom from non-essential code like loading flags or error state

Key Characteristic of Mutation

- Receives external arguments
- Performs asynchronous operations
- **Update other providers** upon completion if needed (does not hold state directly)
- Similar to Redux Toolkit's `async thunk`, but integrated into Provider lifecycle

How it works? State Machine

- Operates internally as a small State Machine
- Automatically transitions through stages from the moment of execution:
 - MutationIdle
 - MutationPending
 - MutationSuccess
 - MutationError

Change in the Question

-  Question: What is the value?
-  Question: What is the current status of this operation?
- Allows for much more precise and independent UI control

Simplifying UI Control

- Making specific buttons show loading (no additional state design needed)
- Easily distinguish between initial loading / adding / deleting state
- Example
 - `Disable` button if pending
 - Show `SnackBar` only on success
 - Change button text from **Submit** to **Retry** on failure

Separation of Data and Action

- Actual Data State → Managed by Notifier / AsyncNotifier
- Mutation do not own the data
- Action Timing & Conditions (Lifecycle) → Managed by Mutation

<provider>

@riverpod

```
class ItemList extends _$ItemList {  
  Future<List<Item>> build() async {}  
  ...
```

```
  Future<void> addItem(String desc) async {  
    state = const AsyncLoading();
```

```
    try {  
      final successData = await addItem(desc);  
      state = AsyncData(successData);  
    } catch (e, st) {  
      state = AsyncError(e, st);  
    }  
  }  
}
```

```
Future<void> updateItem(String id, String desc) async {}
```

```
Future<void> deleteItem(String id) async {}
```

```
  ...
```

```
}
```

<UI>

```
final todoItemState = ref.watch(itemListProvider);  
todoItemState.when(loading, data, error);
```

<provider>

```
// top-level variable
final addItemMutation = Mutation<void>();
final updateItemMutation = Mutation<void>();
final deleteItemMutation = Mutation<void>();

@riverpod
class ItemList extends _$ItemList {
    Future<List<Item>> build() async {}

    ...

    Future<void> addItem(String desc) async {
        final successData = await addItem(desc);
        state = AsyncData(successData);
    }

    ...
}
```

<UI>

```
final addItemState = ref.watch(addItemMutation);
final updateItemState = ref.watch(updateItemMutation);
final deleteItemState = ref.watch(deleteItemMutation);

switch (addItemState) {}

switch (updateItemState) {}

switch (deleteItemState) {}

ElevatedButton(
  onPressed: () {
    addTodoMutation.run(WidgetRef ref, (MutationTransaction tsx) async {
      final notifier = tsx.get(todoListProvider.notifier);
      await notifier.addTodo(_controller.text);
    }
  }
)
```

When to Use, When is it good?

- **Highly Recommended:**
 - When handling multiple actions (Add/Edit/Delete) on one page
 - When you need to display results differently for each action in the UI
 - Shift in thinking from UI-centric to **Business Logic (Action)-centric**

When Notifier Alone is Sufficient

- Single-responsibility tasks like theme changes
- Subscribing to a stream for continuous updates
- Simple value transformations like applying filters
- When a single global loading state is enough for the entire screen

Key Summary

- **Notifier**
 - The Owner of State
- **Mutation**
 - The Manager of Change
- Riverpod 3's message
 - Deliberately separate state and action

Conclusion

- A concept that feels unfamiliar and **challenging** to truly capture through documentation alone.
- Start viewing **Mutation** as an **Operation** rather than a **State**
- **Riverpod 3** isn't just about adding more APIs
- It's an attempt to elevate how we structure our apps

Proving it through Practice

- Theory alone can never let you fully feel the value of **Mutation**
- You probably already know `AsyncNotifier` very well
- However, **we will intentionally build the TODO app once again using the 'uncomfortable' Legacy way**

Why Suffer Again?

- Experience the pain of manually assigning loading states and handling complex UI responses
- **Only by reaching the limits of frustration can you realize the weight of boilerplate that Mutation lifts off your shoulders**
- To go beyond **knowing** and start **feeling**, we begin with the Legacy implementation